

Homework 6

Week 11

1、使用不包含溢出修复功能的程序语言，如何在整型数范围内计算 $\binom{n}{k}$ ，要求计算范围尽可能大

解：

使用递推关系 $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ 进行计算

2、 P 是一个十进制整型数，将 P 划分为 k 段并求各段的乘积，称为 P 的一个 k -乘积

(1)求 1234 的所有 2-乘积

(2)求任意整数 P 的最大 k -乘积 ($P > 10^{k-1}$)

解：

(1) $1 \times 234 = 234, 12 \times 34 = 408, 123 \times 4 = 492$

(2)动态规划： $dp[i, j]$ 表示 P 前 i 位的最大 j -乘积

- 初值：
 - $dp[i, 1]$ 为 P 的前 i 位组成的整数
 - $dp[i, j] = 0, i < j$
- 迭代：

$$dp[i, j] = \max_{1 \leq k \leq i-1} \{dp[k, j-1] \times \overline{k+1..i}\}$$

其中 $\overline{k+1..i}$ 表示 P 的第 $k+1$ 高位到第 i 高位组成的整数

- 问题解： $dp[n, k]$

3、求 $1 \oplus 2 \oplus 3 \oplus \dots \oplus N$ 的值，其中 $\oplus$ 表示异或，所有数字实际均用二进制码表示， N 为输入，要求计算量尽可能小

解：

- 记 $f(N) = 1 \oplus 2 \oplus 3 \oplus \dots \oplus N$
- 则有：
 - 若 $N \% 4 == 0$ ，则 $f(N) = N$
 - 若 $N \% 4 == 1$ ，则 $f(N) = 1$
 - 若 $N \% 4 == 2$ ，则 $f(N) = N + 1$
 - 若 $N \% 4 == 3$ ，则 $f(N) = 0$
- 时空复杂度均为 $O(1)$

4、将正整数 n 表示为 $n = n_1 + n_2 + \dots + n_k (n_1 \geq n_2 \geq \dots \geq n_k)$ ，称为 n 的一个划分
例如 4 的所有划分为 4, $3 + 1$, $2 + 2$, $2 + 1 + 1$, $1 + 1 + 1 + 1$ 共 5 种

(1)求 6 的所有划分

(2)求 n 的划分种类数，要求时间复杂度 $O(n^2)$

解：

(1) $6, 5 + 1, 4 + 2, 4 + 1 + 1, 3 + 3, 3 + 2 + 1, 3 + 1 + 1 + 1, 2 + 2 + 2, 2 + 2 + 1 + 1, 2 + 1 + 1 + 1 + 1, 1 + 1 + 1 + 1 + 1 + 1$ 共 11 种

(2)动态规划： $q(n, m)$ 表示 n 的所有划分中最大数不超过 m 的种类数

- 初值：
 - $q(n, m) = q(n, n) (n < m)$ ，因为 n 的划分中最大数一定不超过 n ，也一定不超过 m
 - $\forall n \in \mathbb{N}_+, q(n, 1) = 1$ ，因为只有 n 个 1 相加 1 种划分
- 迭代： $q(n, m) = q(n, m - 1) + q(n - m, m)$
 - $q(n, m - 1)$ 表示最大数不超过 $m - 1$ 的划分种类数，此时所有划分一定不含 m
 - $q(n - m, m)$ 表示至少含有 1 个 m 的划分种类数， n 中去除 m 后剩余 $n - m$ ，而对 $n - m$ 的划分既可以包含 m 也可以不包含 m ，但最大数不能超过 m
 - “划分中只有 $< m$ 的数”和“划分中包含至少 1 个 m ，且其余部分的划分中最大数不超过 m ”2 种情况并集的规模即为 $q(n, m)$
- 问题解： $q(n, n)$
- 时间复杂度： $q[n][n]$ 数组的每个元素均可在 $O(1)$ 时间内计算得到，因此总复杂度 $O(n^2)$

Week 12

1、区间套问题：给定若干区间，求这些区间中包含于其他区间的区间

例如 $(1, 9), (2, 8), (3, 10), (4, 7), (5, 9)$ 的解为 $(2, 8), (4, 7), (5, 9)$

要求时间复杂度 $O(n \log n)$ ，要求使用归约求解

解：

定义**极点**为： $\forall j \neq i, (x_j > x_i) \vee (y_j < y_i)$

则可根据极大点问题设计算法求出所有极点，而本问题解为所有**非极点**

2、证明：若存在时间复杂度为 $O(T(n))$ 的 $n \times n$ 下三角矩阵乘法算法，则存在时间复杂度为 $O(T(n) + n^2)$ 的 $n \times n$ 任意矩阵乘法算法

解：

可根据

$$\begin{pmatrix} O & O & O \\ A & O & O \\ A & A & O \end{pmatrix} \times \begin{pmatrix} O & O & O \\ B & O & O \\ B & B & O \end{pmatrix} = \begin{pmatrix} O & O & O \\ O & O & O \\ AB & O & O \end{pmatrix}$$

推导复杂度关系

3、证明：*Graham* 算法（时间复杂度 $O(n \log n)$ ）是凸包问题的最优算法之一

解：

将排序问题归约为凸包问题

对于排序问题中的元素 x_i ，构造点 (x_i, x_i^2)

若 $o(n \log n)$ 时间内可以求解凸包问题，则 $o(n \log n) + O(n) = o(n \log n)$ 时间内可以求解排序问题

而排序问题的时间复杂度 $\Omega(n \log n)$ ，因此凸包问题的时间复杂度 $\Omega(n \log n)$

4、证明：基于比较构建二叉搜索树需要 $\Omega(n \log n)$ 时间

解：

二叉搜索树的中序遍历结果为有序序列，时间复杂度 $\Theta(n)$

若 $o(n \log n)$ 时间内可以构建二叉搜索树，则 $o(n \log n) + O(n) = o(n \log n)$ 时间内可以求解排序问题

而排序问题的时间复杂度 $\Omega(n \log n)$ ，因此构建二叉搜索树的时间复杂度 $\Omega(n \log n)$